Universidad de La Sabana



Algoritmos para grafos

Árboles de recubrimiento mínimo y caminos más cortos

Facultad de Ingeniería · Estructura de Datos y Algoritmos

Clase 2

Árboles de recubrimiento mínimo



Universidad de La Sabana

Grafos en GCL: notación

En los pseudocódigos de esta clase y la siguiente, los grafos se representan **solo con arreglos**:

Lista de adyacencia (Prim, Dijkstra)

```
con n : nat                          {vértices 0, …, n-1}
con grado : array[0,n) of nat
con ady : array[0,n)x[0,n) of nat
con c : array[0,n)x[0,n) of int
```

ady[u,i] es el i-ésimo vecino de u (0 ≤ i < grado[u]) y c[u,i] el costo de esa arista.

Lista de aristas (Kruskal, Bellman-Ford)

```
con m : nat                          {aristas 0, …, m-1}
con eu, ev : array[0,m) of nat
con ec : array[0,m) of int
```

La arista i va de eu[i] a ev[i] y cuesta ec[i].

`con INF : int` representa ∞: un valor mayor que el costo de cualquier camino. Antes de sumar se verifica que el costo no sea INF.

Algoritmos voraces (greedy)

En cada paso se elige la opción que **parece mejor localmente**, con la esperanza de llegar a una solución **global óptima**. Nunca se deshace una decisión.

¿Cuándo falla?

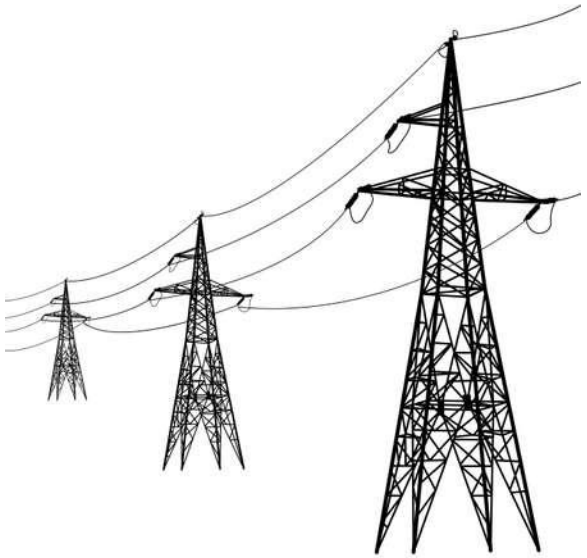
¿Puede dar un ejemplo donde la estrategia voraz no da la respuesta correcta?

¿Cuándo funciona?

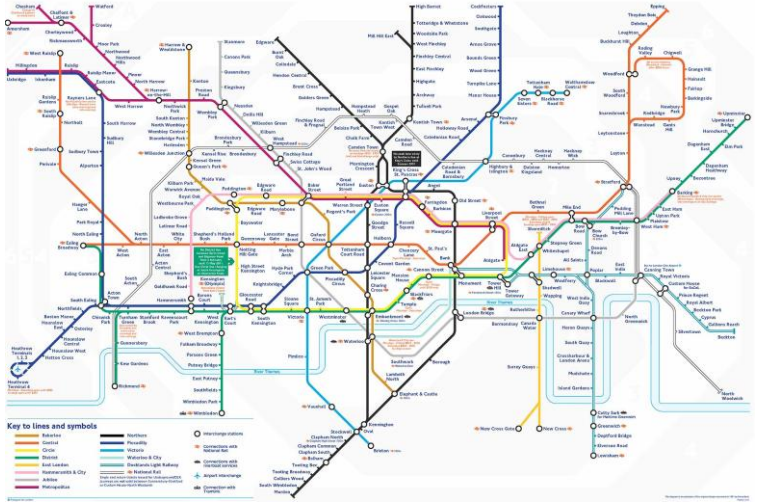
En problemas de gran interés como el **árbol de recubrimiento mínimo** (MST, minimum spanning tree): ahí la estrategia voraz sí es óptima.

Árboles de recubrimiento

- También llamados árboles **de cubrimiento**, **generadores** o **spanning trees**.
- Aparecen al diseñar redes: conectar todos los puntos sin conexiones redundantes.



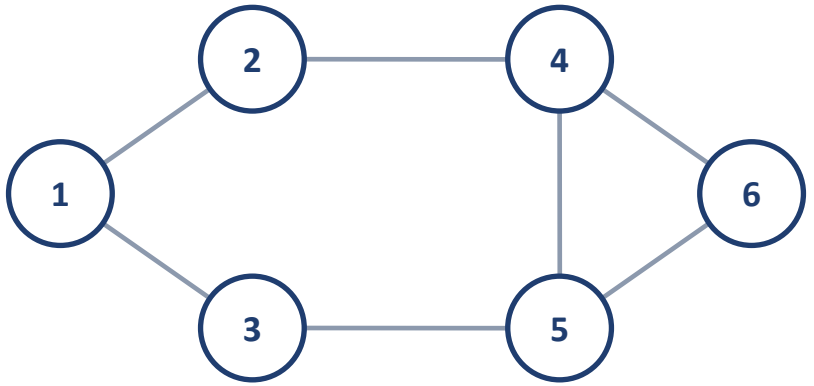
Red eléctrica



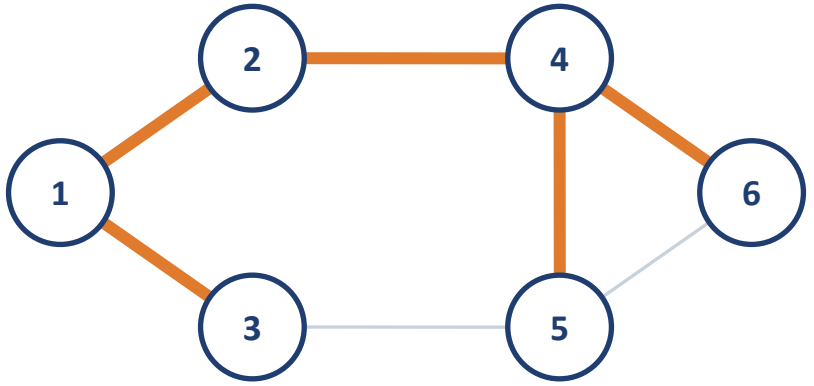
Red de metro

Definición: árbol de recubrimiento

- Dado un grafo G = (V, E) **no dirigido y conectado**, un árbol de recubrimiento es un subgrafo que contiene **todos los vértices** de G y **es un árbol**.
- Si |V| = n, el árbol tiene **n – 1 aristas**.
- ¿Cuántos árboles de recubrimiento tiene un grafo? A lo sumo **C(m, n – 1)**: no todo subconjunto de n − 1 aristas sirve, porque los que forman ciclo no son árboles.
- Grafo completo: $\mathbf{n^{n-2}}$ árboles (fórmula de Cayley).



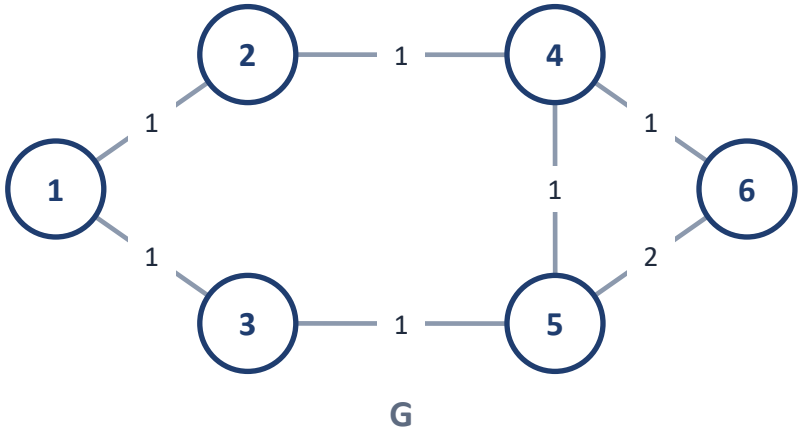
G: n = 6, m = 7



Un árbol de recubrimiento de G

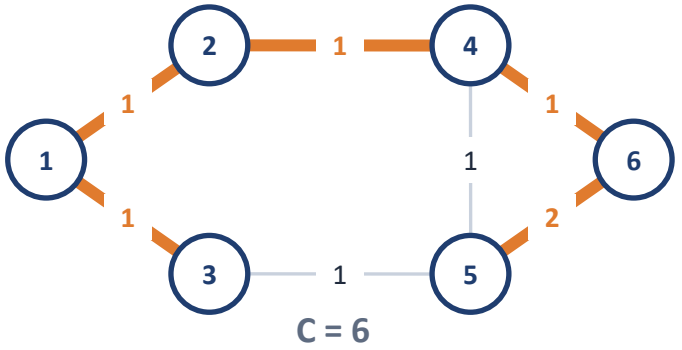
Árbol de recubrimiento mínimo (MST)

Problema: en un grafo con pesos, hallar el árbol de recubrimiento de **costo total mínimo**.

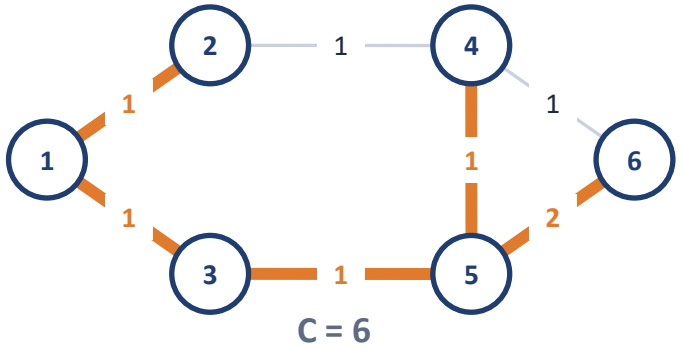


G

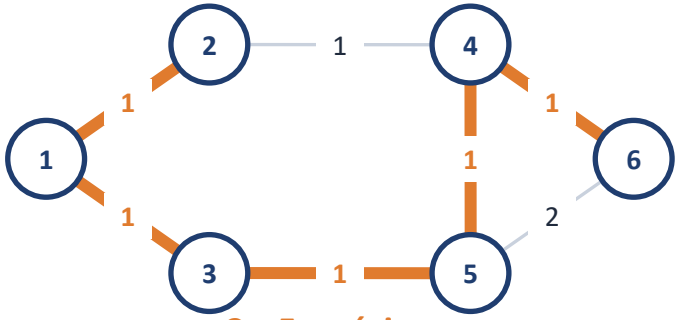
¡Puede haber **más de un** árbol de recubrimiento mínimo!



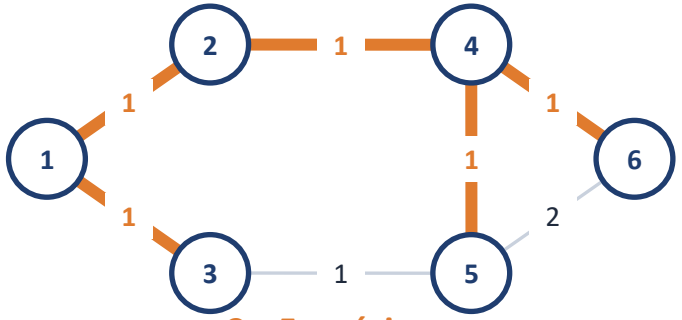
C = 6



C = 6



C = 5 · mínimo



C = 5 · mínimo

¿Por qué estudiar el MST?

Aplicaciones

Tiene miles de aplicaciones.

Voraz y óptimo

Uno de los pocos problemas donde un algoritmo voraz da la solución óptima.

Fácil de implementar

Pocas líneas de código con las estructuras correctas.

Diseño de redes

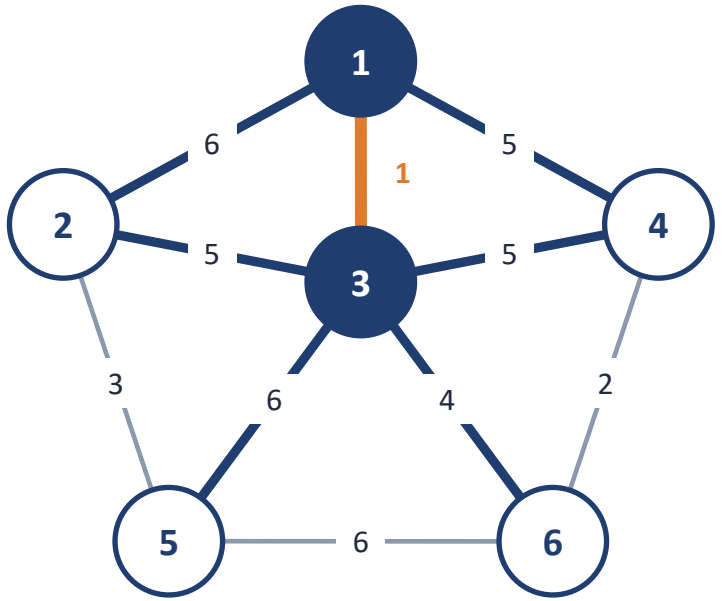
Planeamiento de redes de distribución y de comunicación.

Clustering

Es la base de una estrategia de agrupamiento de datos.

Algoritmo de Prim

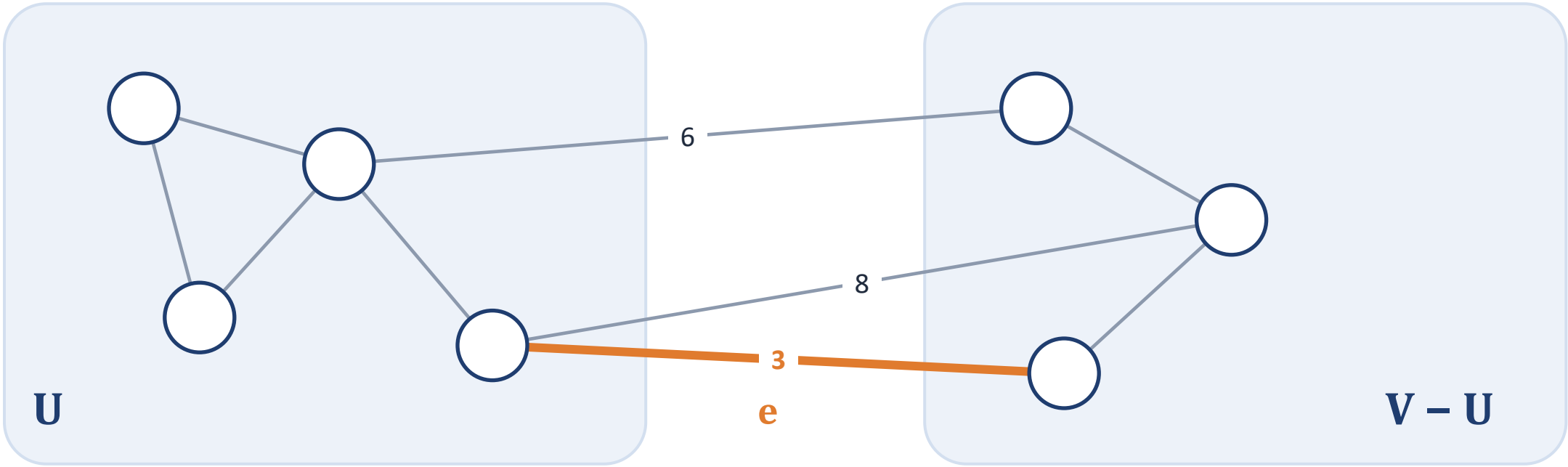
- Comienza en **un vértice** y hace crecer el árbol **una arista a la vez**.
- Como algoritmo voraz, ¿qué arista elegir?
- La **más liviana** que agrega un vértice nuevo al árbol, es decir, que no crea un ciclo.



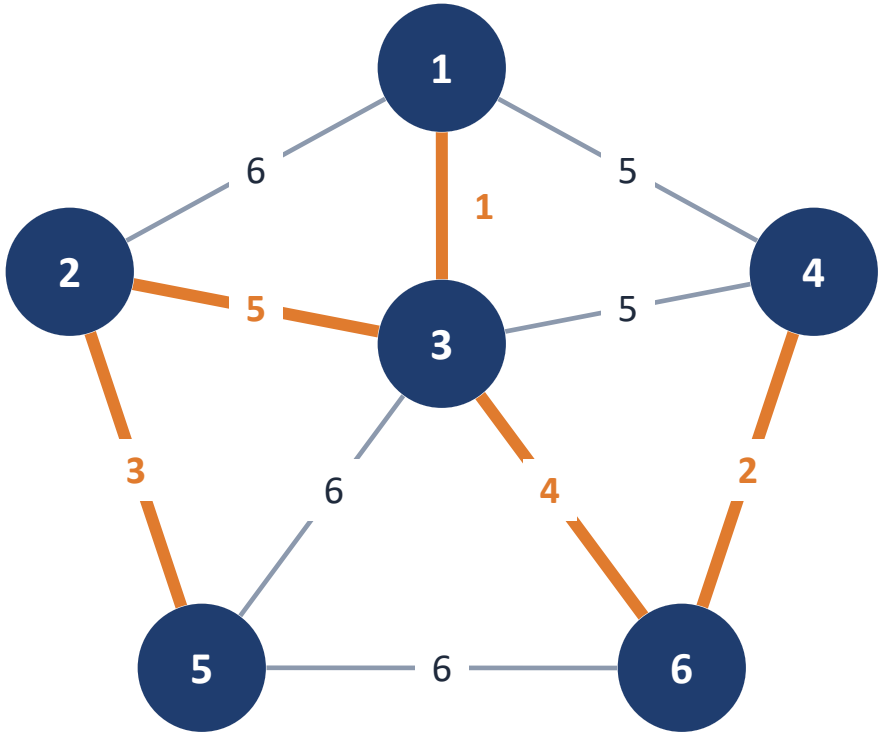
Árbol actual: {1, 3}. Las aristas en azul salen del árbol; Prim toma la más liviana: (3, 6) con costo 4.

¿Por qué funciona? Propiedad del corte

Sea G = (V, E) conectado con pesos y U ⊂ V. Si e = (u, v) es la arista **de menor costo** con u ∈ U y v ∈ V − U, entonces **existe un MST que incluye a e**.



Prim paso a paso (desde 1)



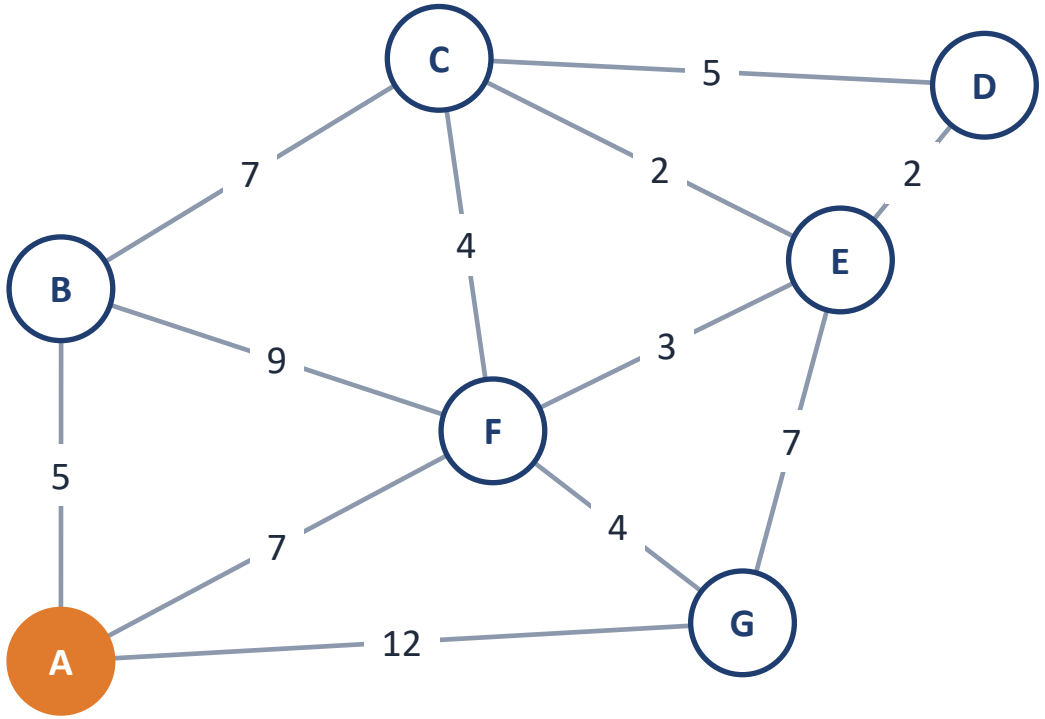
Fin: n − 1 = 5 aristas. Costo total del árbol: 15.

Aristas del árbol: (1, 3) 1 · (3, 6) 4 · (6, 4) 2 · (3, 2) 5 · (2, 5) 3

Costo acumulado: 15

Azul: aristas que salen del árbol · Naranja: aristas del árbol y último vértice agregado

Ejercicio: Prim desde A

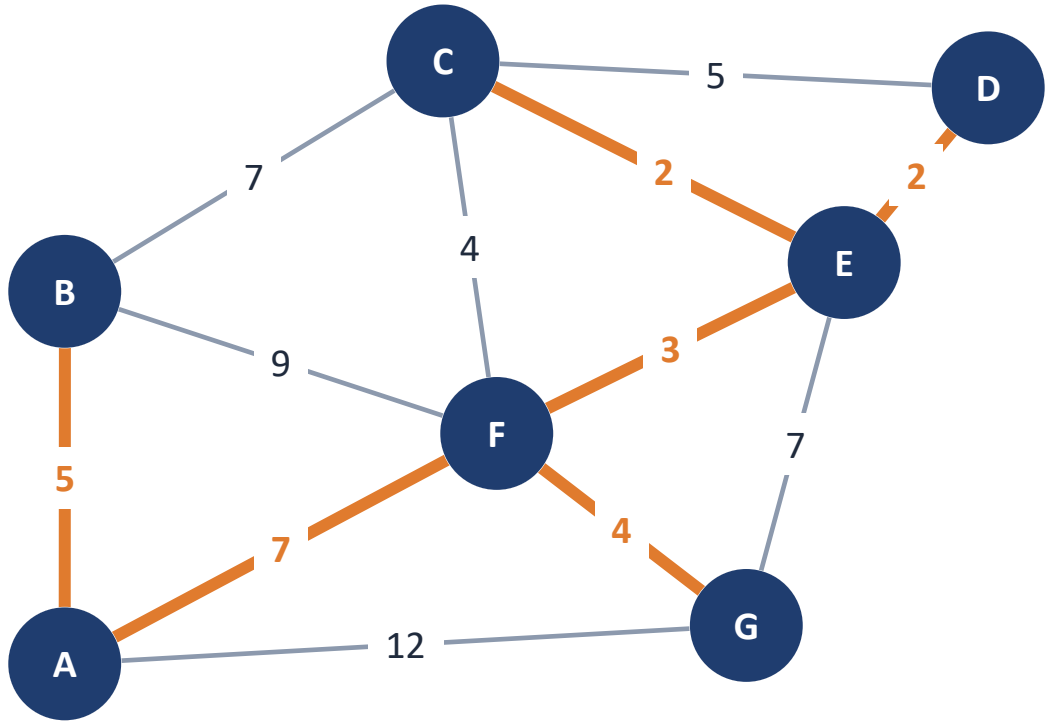


Aplique Prim partiendo de A

Anote en qué orden entran los vértices al árbol y el costo total.

¿Hay algún paso donde dos aristas empaten?

Solución: Prim desde A



Azul: aristas que salen del árbol · Naranja: aristas del árbol

Fin: n – 1 = 6 aristas. Costo total del árbol: 23.

Aristas del árbol: (A, B) 5 · (A, F) 7 · (F, E) 3 · (E, C) 2 · (E, D) 2 · (F, G) 4

Costo acumulado: 23

Prim en GCL

```
{Pre: G conectado y no dirigido}
var arbol : array[0,n) of bool
var costo, padre : array[0,n) of int
var k, u, v, i, total : int
v := 0;
do v < n → arbol[v], costo[v], padre[v] := false, INF, -1;
           v := v + 1
od;
costo[0], total, k := 0, 0, 0;
do k < n →
   u := minimo();
   arbol[u], total, k, i := true, total + costo[u], k + 1, 0;
   do i < grado[u] →            {actualiza los vecinos de u}
      v := ady[u,i];
      if ¬arbol[v] ∧ c[u,i] < costo[v] →
            costo[v], padre[v] := c[u,i], u
      [] arbol[v] ∨ c[u,i] ≥ costo[v] → skip
      fi;
      i := i + 1
   od
od
{Post: (padre[v], v), v ≠ 0, son las aristas de un MST}
```

```
fun minimo ( ) : int      {fuera del árbol}
   var u, v, mejor : int
   u, mejor, v := -1, INF, 0;
   do v < n →
      if ¬arbol[v] ∧ costo[v] < mejor →
            u, mejor := v, costo[v]
      [] arbol[v] ∨ costo[v] ≥ mejor → skip
      fi;
      v := v + 1
   od
ret u
```

Este pseudocódigo

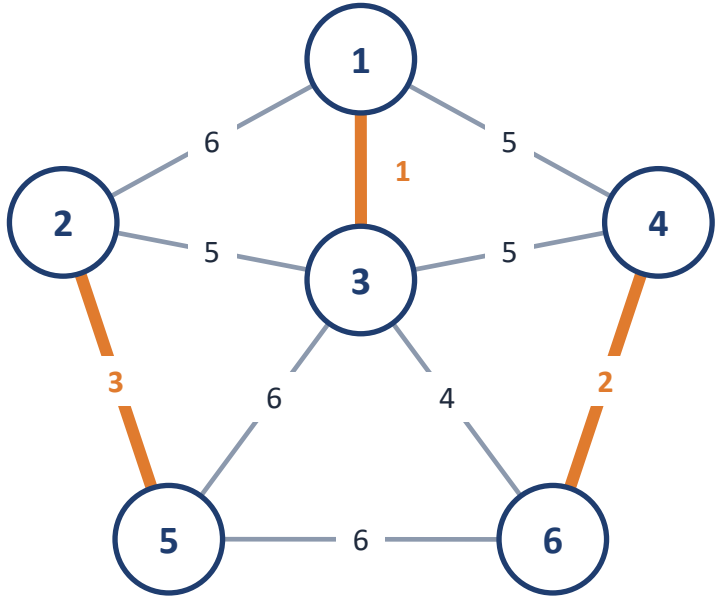
O(n²)

Con heap

O(m log n)

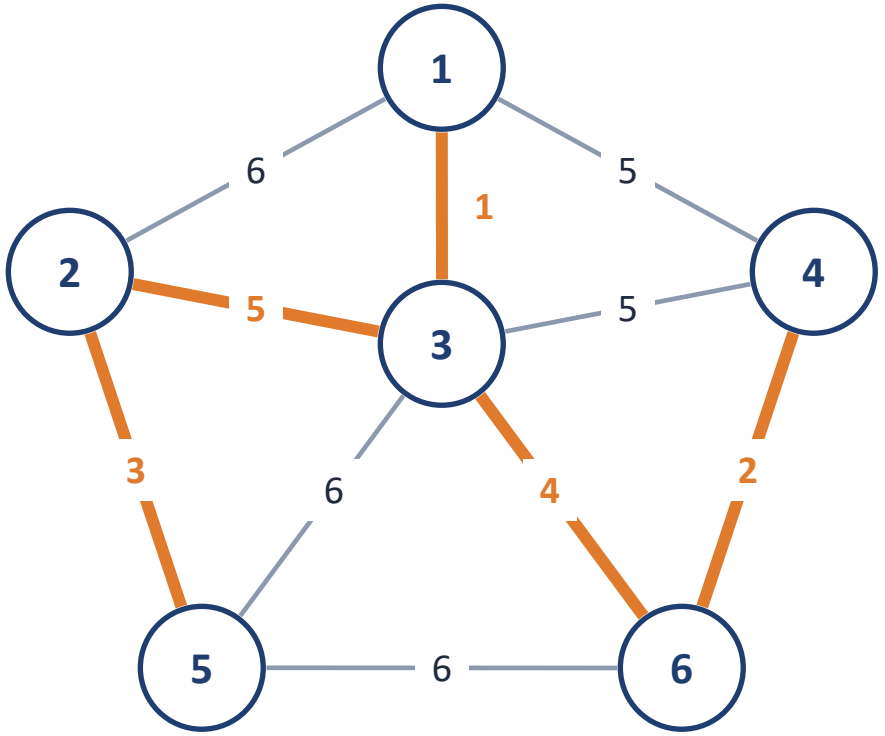
Algoritmo de Kruskal

- También es voraz, pero **no parte de un vértice**.
- Ordena las aristas por costo y las agrega de la más liviana a la más pesada, **siempre que no formen un ciclo**.
- Mientras avanza, lo que tiene es un **bosque** que se va uniendo hasta formar un solo árbol.



Después de 3 pasos: un bosque con tres árboles.

Kruskal paso a paso

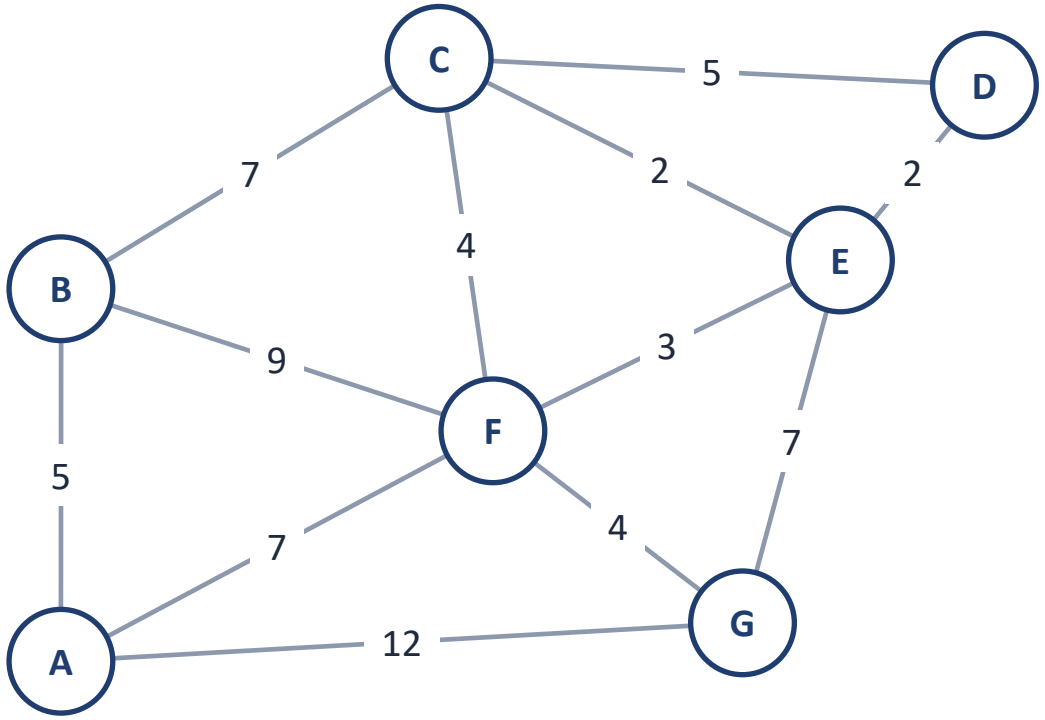


Aristas ordenadas por costo

(1, 3)	1	✓
(4, 6)	2	✓
(2, 5)	3	✓
(3, 6)	4	✓
(1, 4)	5	✗ ciclo
(2, 3)	5	✓
(3, 4)	5	
(1, 2)	6	
(3, 5)	6	
(5, 6)	6	

Fin: ya hay n − 1 = 5 aristas, el resto no hace falta revisarlas. Costo total: 15.

Ejercicio: Kruskal

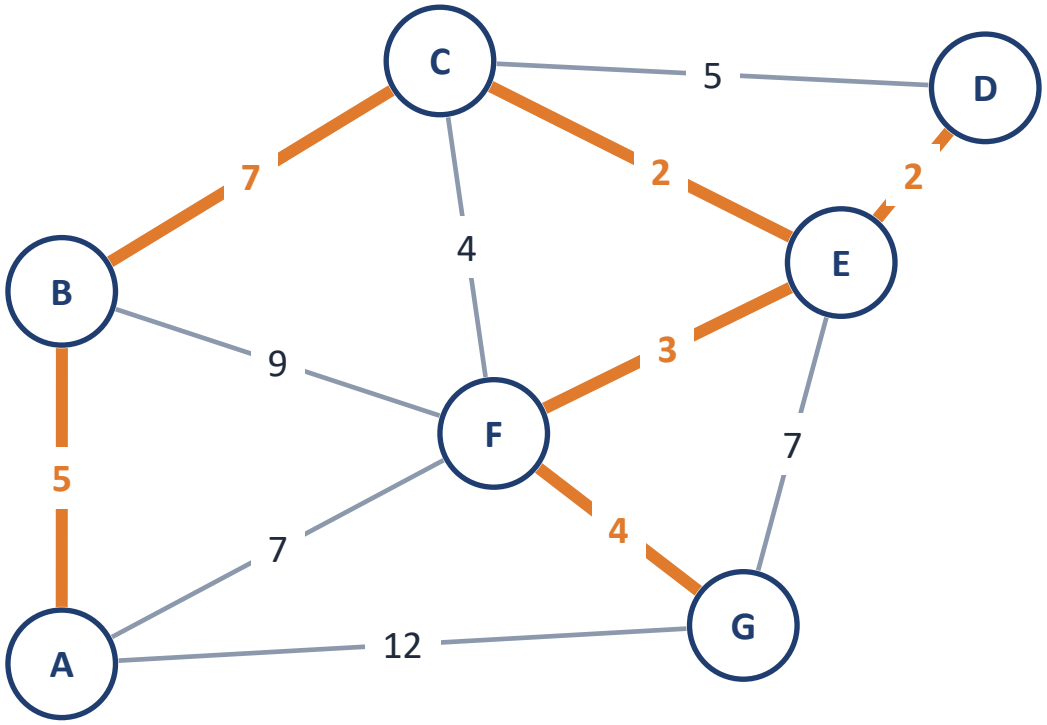


Aplique Kruskal sobre G

No necesita vértice inicial.

Anote qué aristas acepta, cuáles descarta y el costo total. ¿Obtiene el mismo árbol que con Prim?

Solución: Kruskal



Aristas ordenadas por costo

(C, E)	2	✓
(D, E)	2	✓
(F, E)	3	✓
(C, F)	4	✗ ciclo
(F, G)	4	✓
(A, B)	5	✓
(C, D)	5	✗ ciclo
(B, C)	7	✓
(A, F)	7	
(E, G)	7	
(B, F)	9	
(A, G)	12	

Fin: ya hay n − 1 = 6 aristas, el resto no hace falta revisarlas. Costo total: 23.

Kruskal en GCL

```
{Pre: aristas ordenadas: ec[0] ≤ ec[1] ≤ … ≤ ec[m-1]}
var enA : array[0,m) of bool      {¿la arista i queda en el MST?}
var padre, tam : array[0,n) of nat
var i, v : nat
var total : int
v := 0;
do v < n → padre[v], tam[v] := v, 1; v := v + 1 od;
i, total := 0, 0;
do i < m →
   if find(eu[i]) ≠ find(ev[i]) → enA[i], total := true, total + ec[i];
                                  union(eu[i], ev[i])
   [] find(eu[i]) = find(ev[i]) → enA[i] := false     {formaría ciclo}
   fi;
   i := i + 1
od
{Post: las aristas i con enA[i] forman un MST de costo total}
```

- No requiere un vértice de inicio.
- find(x) devuelve el representante del árbol que contiene a x: la arista forma ciclo ⇔ sus dos extremos tienen el mismo representante.
- find y union: siguiente diapositiva.

Ordenar + m operaciones

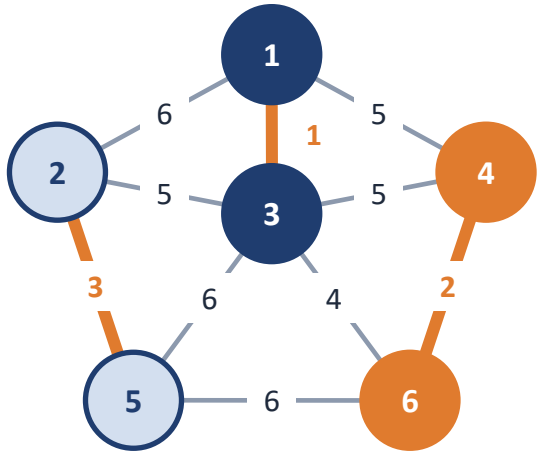
O(m log m)

Union-Find: la idea

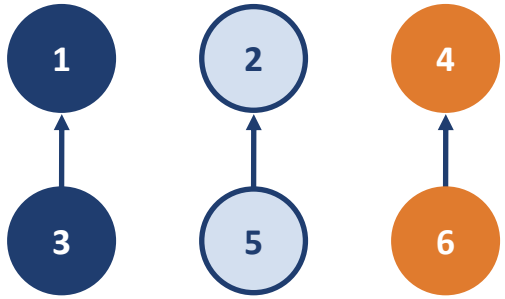
- Kruskal pregunta una y otra vez: **¿u y v ya están conectados?**
- Union-Find guarda cada grupo como un **árbol**. La raíz es el **representante** del grupo y `padre[v]` apunta un paso hacia ella.
- **find(v)**: seguir las flechas desde v hasta la raíz.
- Misma raíz ⇔ mismo grupo ⇔ la arista formaría un **ciclo**.
- **union**: hacer que una raíz apunte a la otra.

¿La arista (3, 6) forma ciclo? find(3) = 1 y find(6) = 4: raíces distintas, así que **no**: Kruskal la acepta.

Kruskal después de aceptar (1, 3), (4, 6) y (2, 5)



Los mismos tres grupos en Union-Find (flecha = padre)



Union-Find en GCL

```
fun find ( x : nat ) : nat
   var r : nat
   r := x;
   do padre[r] ≠ r → r := padre[r] od
ret r

proc union ( u, v : nat )          {unión por tamaño}
   var a, b : nat
   a, b := find(u), find(v);
   if tam[a] < tam[b] → padre[a], tam[b] := b, tam[a] + tam[b]
   [] tam[a] ≥ tam[b] → padre[b], tam[a] := a, tam[a] + tam[b]
   fi
```

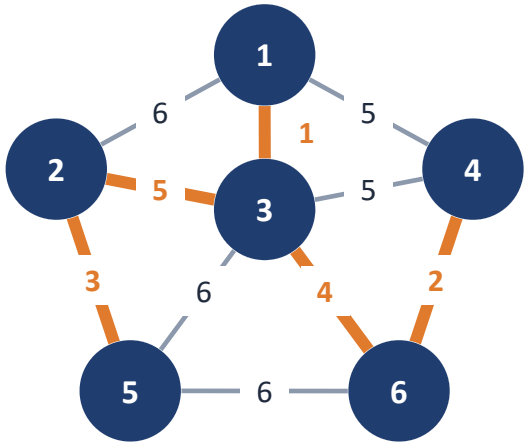
- Cada conjunto es un árbol: padre[x] apunta hacia la raíz, que es el **representante**.
- **Unión por tamaño**: el árbol pequeño se cuelga del grande, así la altura es a lo sumo log n y find cuesta **O(log n)**.
- Con **compresión de caminos** (apuntar todo el camino a la raíz) cada operación queda prácticamente en O(1).

Kruskal con Union-Find

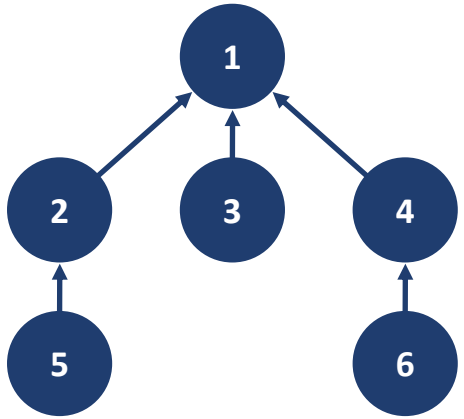
O(m log m)

Union-Find paso a paso

Grafo de Kruskal



Bosque Union-Find (flecha = padre[v])



Reglas

find(v): seguir las flechas hasta la raíz.
Misma raíz ⇔ ya conectados ⇔ ciclo.
union: la raíz del grupo pequeño apunta a la del grande.

v	1	2	3	4	5	6
padre	1	1	1	1	2	4
tam	6	2	1	2	1	1

Fin

Un solo grupo con raíz 1: Kruskal ya tiene sus 5 aristas.
Altura del árbol: $2 \le \log_2 6$.
Por eso find recorre pocas flechas.

Prim vs Kruskal

	Complejidad	Conviene en
Prim con arreglo	$O(n^2)$	Grafos densos ($m \approx n^2$): no depende de m
Prim con heap	O(m log n)	Grafos dispersos
Kruskal con Union-Find	O(m log m)	Grafos dispersos ($m \approx n$)

Tarea 3 – Freckles

Hasta 100 pecas y cualquier par se puede unir con una línea: es un grafo completo ($m \approx n^2/2$). ¿Cuál usaría?

Clase 3

Caminos más cortos



Universidad de La Sabana

Caminos más cortos: aplicaciones

Encontrar el camino más corto entre dos nodos es necesario en muchas aplicaciones:

Transporte

Rutas en mapas, logística, navegación GPS.

Movimiento robótico

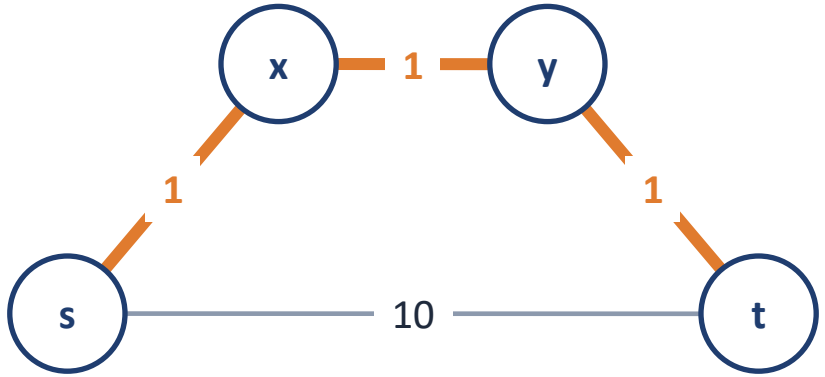
Planear la trayectoria de un robot evitando obstáculos.

Comunicación

Enrutar paquetes en una red con la menor latencia.

Grafos no ponderados: basta BFS

- Sin pesos, el camino más corto es el de **menos aristas**: BFS lo encuentra en **O(n + m)**.
- ¿Sirve BFS con pesos? **No**: a veces recorrer más aristas cuesta menos.
- ¿Pesos negativos? Dijkstra exige pesos **≥ 0**: con una sola arista negativa puede fallar, aunque no haya ciclos negativos (lo veremos).



BFS elige s → t (1 arista, costo 10). El camino más corto es s → x → y → t, con costo 3.

Algoritmo de Dijkstra: la idea

- Si (s, …, x, …, t) es el camino más corto de s a t, entonces (s, …, x) es el camino más corto de s a x. **¡Tiene la forma de programación dinámica!**
- d(s, s) = 0. Las distancias ya conocidas se usan para encontrar las de nodos más lejanos.

1

Escoger

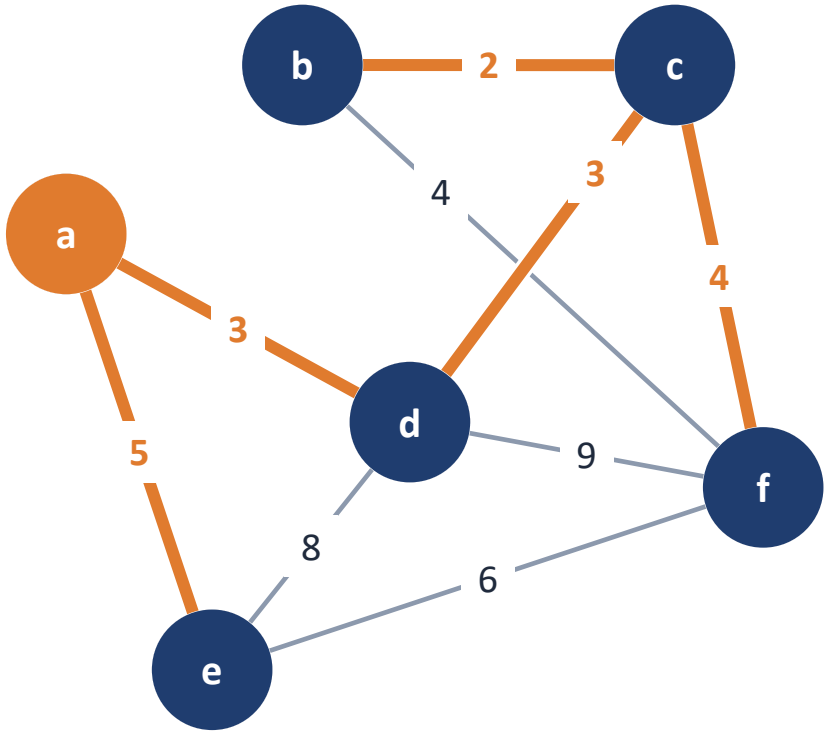
El vértice w no escogido al que se llega con costo mínimo desde los ya escogidos.

2

Actualizar

Los costos de los vértices no escogidos, usando w como posible vértice intermedio.

Dijkstra paso a paso

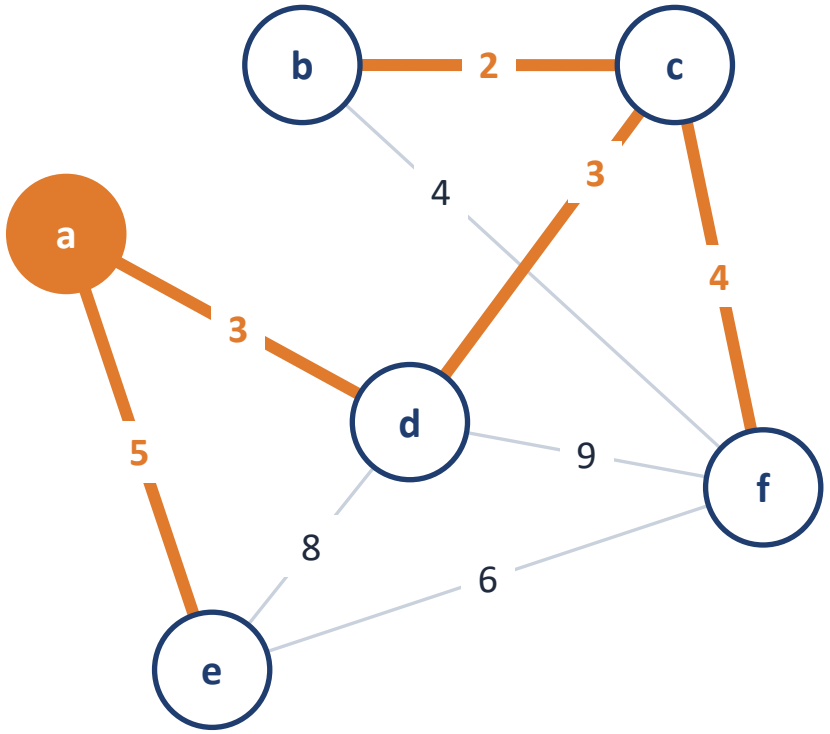


Destino	Camino	Costo
b	⟨a, d, c, b⟩	8
c	⟨a, d, c⟩	6
d	⟨a, d⟩	3
e	⟨a, e⟩	5
f	⟨a, d, c, f⟩	10

Fin: todos los vértices alcanzables fueron escogidos. Las aristas naranja forman el árbol de caminos más cortos.

Naranja: vértice escogido · Azul oscuro: ya escogidos · Aristas azules: las que se relajan · Aristas naranja: mejor camino conocido

La estructura resultante



- Un **árbol de caminos más cortos** con raíz en a.
- Contiene el camino más corto de a a **cualquier** otro vértice.
- Tiene n vértices y **n – 1 aristas**.
- ¿En qué se diferencia de un MST?

Dijkstra en GCL

```
{Pre: c[u,i] ≥ 0 para toda arista}
con s : nat
var d : array[0,n) of int
var listo : array[0,n) of bool
var k, u, v, i : int
v := 0;
do v < n → d[v], listo[v] := INF, false; v := v + 1 od;
d[s], k := 0, 0;
do k < n →
   u := minimo();
   if u = -1 → k := n            {el resto es inalcanzable}
   [] u ≠ -1 →
        listo[u], k, i := true, k + 1, 0;
        do i < grado[u] →
           v := ady[u,i];
           if d[u] + c[u,i] < d[v] → d[v] := d[u] + c[u,i]
           [] d[u] + c[u,i] ≥ d[v] → skip
           fi;
           i := i + 1
        od
   fi
od
{Post: d[v] = costo mínimo de s a v; INF si no se alcanza}
```

```
fun minimo ( ) : int      {no listo, d mínimo}
   var u, v, mejor : int
   u, mejor, v := -1, INF, 0;
   do v < n →
      if ¬listo[v] ∧ d[v] < mejor →
            u, mejor := v, d[v]
      [] listo[v] ∨ d[v] ≥ mejor → skip
      fi;
      v := v + 1
   od
ret u
```

Este pseudocódigo

O(n²)

Con heap

O((n+m) log n)

Dijkstra con cola de prioridad

```
con s : nat
var d : array[0,n) of int
var H : set of int × nat            {montículo de pares (costo, vértice)}
var du, u, v, i : int
v := 0;
do v < n → d[v] := INF; v := v + 1 od;
d[s], H := 0, {(0, s)};
do H ≠ ∅ →
   extraerMin(H, du, u);             {saca el par de menor costo}
   if du > d[u] → skip               {par viejo: se ignora}
   [] du = d[u] →
        i := 0;
        do i < grado[u] →
           v := ady[u,i];
           if du + c[u,i] < d[v] → d[v] := du + c[u,i];
                                   insertar(H, d[v], v)
           [] du + c[u,i] ≥ d[v] → skip
           fi; i := i + 1
        od
   fi
od
```

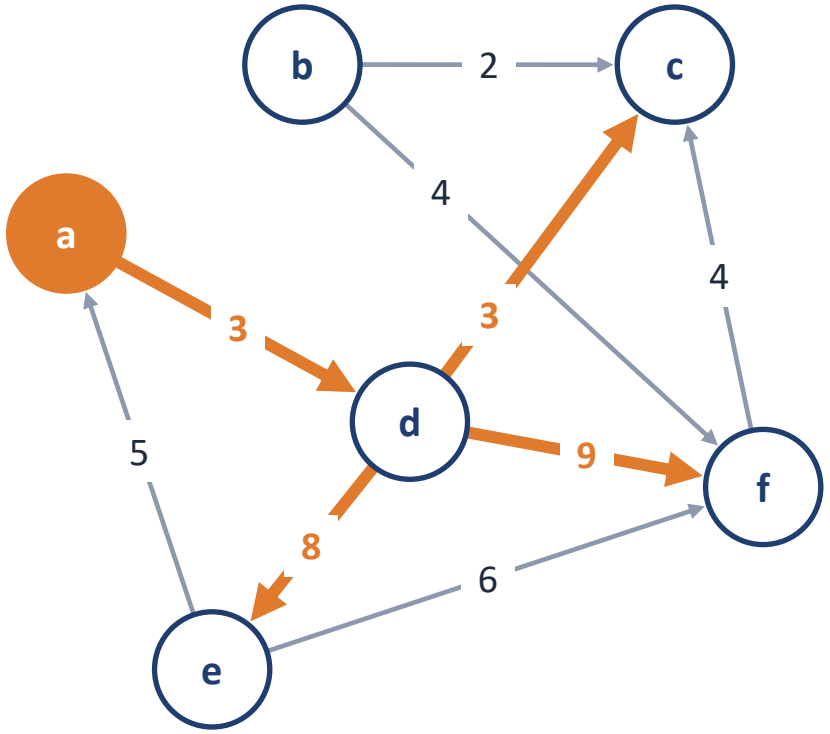
- extraerMin e insertar cuestan O(log n) en un montículo.
- No se puede bajar la prioridad de un par: se inserta uno nuevo y los viejos se descartan al sacarlos.
- **Tarea 3 – Email**: n = 20 000, m = 50 000 → $n^2 = 4 \cdot 10^8$ frente a $(n + m) \log n \approx 10^6$.

Complejidad

$O((n+m) \log n)$

¿Y si el grafo es dirigido?

Dijkstra también funciona en grafos dirigidos, pero la respuesta puede ser distinta a la del grafo no dirigido correspondiente.



Destino	Camino	Costo
b	–	∞
c	⟨a, d, c⟩	6
d	⟨a, d⟩	3
e	⟨a, d, e⟩	11
f	⟨a, d, f⟩	12

b es inalcanzable desde a. e y f cuestan más que en el grafo no dirigido (5 y 10).

Caminos más cortos en el análisis de redes

Muchas preguntas sobre redes reales se responden calculando caminos más cortos:

1

Rutas

¿Cuál es el camino más rápido o más barato? Navegación, logística, transporte público.

2

Enrutamiento

Cada router de Internet decide por dónde enviar los paquetes calculando caminos más cortos (OSPF).

3

Centralidad

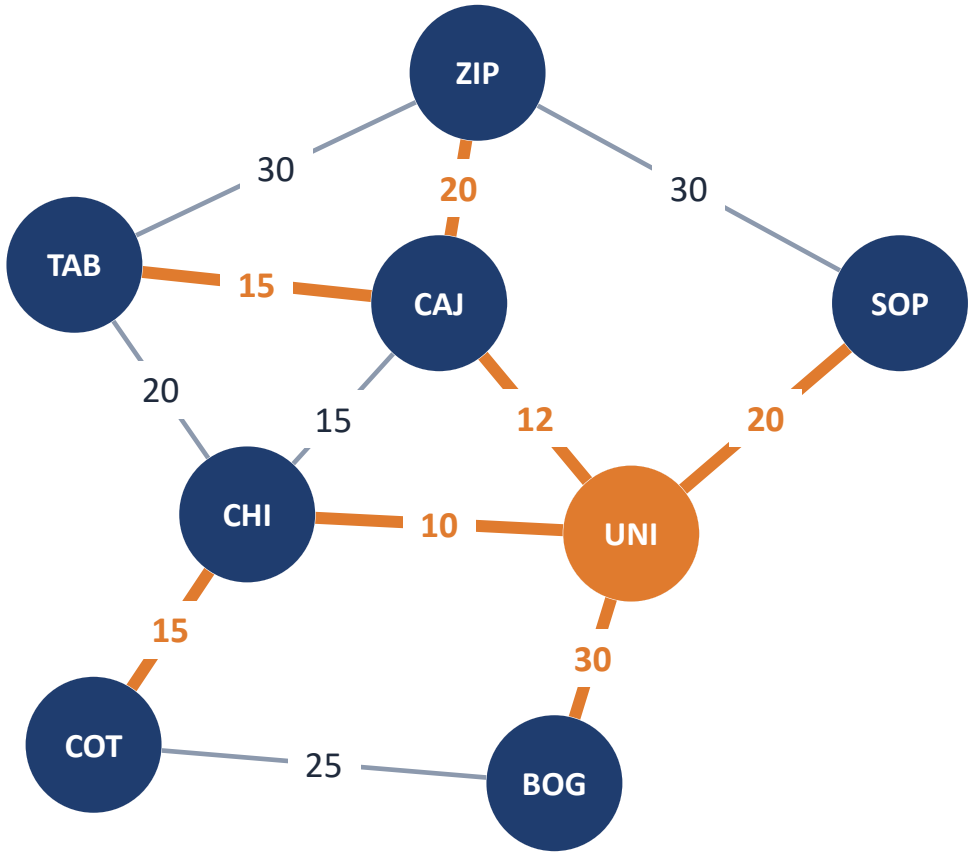
¿Quién está mejor ubicado en la red? Se mide con las distancias a todos los demás.

4

Mundo pequeño

¿A cuántos pasos está cualquier persona de cualquier otra en una red social?

Ejemplo: ¿cómo llego desde La Sabana?



Destino	Ruta	Min
BOG	⟨UNI, BOG⟩	30
CAJ	⟨UNI, CAJ⟩	12
CHI	⟨UNI, CHI⟩	10
COT	⟨UNI, CHI, COT⟩	25
SOP	⟨UNI, SOP⟩	20
TAB	⟨UNI, CAJ, TAB⟩	27
ZIP	⟨UNI, CAJ, ZIP⟩	32

Fin: todos los vértices alcanzables fueron escogidos. Las aristas naranja forman el árbol de caminos más cortos.

UNI Universidad de La Sabana · BOG Bogotá (Portal Norte) · COT Cota · CHI Chía · CAJ Cajicá · TAB Tabio · ZIP Zipaquirá · SOP Sopó. Tiempos ilustrativos, en minutos.

Enrutamiento en Internet (OSPF)

- Cada enlace tiene un **costo**: más ancho de banda, menor costo.
- Cada router conoce el mapa completo y corre **Dijkstra desde sí mismo**.
- De su árbol de caminos más cortos sale la **tabla de enrutamiento**: para cada destino, a qué vecino enviar el paquete.
- Si un enlace falla, se recalcula.

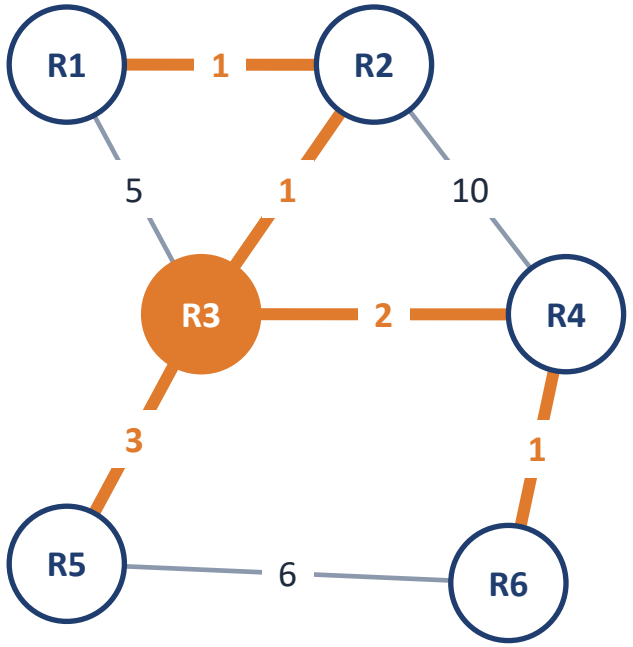
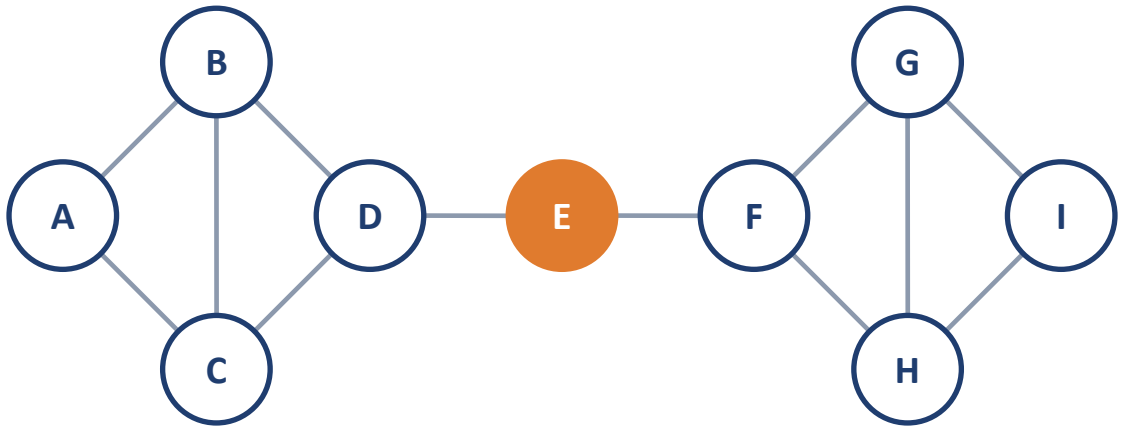


Tabla de R3

Destino	Costo	Siguiente salto
R1	2	**R2**
R2	1	R2
R4	2	R4
R5	3	R5
R6	3	R4

R1 es vecino directo de R3 (costo 5), pero el tráfico va por R2: costo 2.

Centralidad: ¿quién es importante en una red?



- **Cercanía** de v: (n − 1) / suma de distancias de v a los demás. Más alta = más cerca de todos.
- **Intermediación** de v: cuántos pares de personas tienen su camino más corto pasando por v.
- Ambas necesitan las distancias entre todos los pares: n recorridos (BFS sin pesos, Dijkstra con pesos).

Persona	Amigos	Cercanía	Intermediación
A	2	0,30	0
B	3	0,38	3
C	3	0,38	3
D	3	0,47	15
E	2	0,50	16
F	3	0,47	15
G	3	0,38	3
H	3	0,38	3
I	2	0,30	0

E tiene solo 2 amigos, pero es el **puente** entre los dos grupos: es la persona más central.

Mundo pequeño: ¿a cuántos pasos estamos?

≈ 6

Milgram, 1967

Cartas que pasaban de mano en mano entre desconocidos en EE. UU.

4,74

Facebook, 2011

Distancia promedio entre 721 millones de usuarios.

3,57

Facebook, 2016

Distancia promedio entre 1.590 millones de usuarios.

¿Cómo se calcula con 1.600 millones de vértices?

La distancia promedio exacta exige caminos más cortos entre todos los pares: unos 10^{18} pares. Ni con BFS desde cada vértice alcanza el tiempo; se usaron algoritmos aproximados basados en BFS (HyperANF).

En la práctica: NetworkX

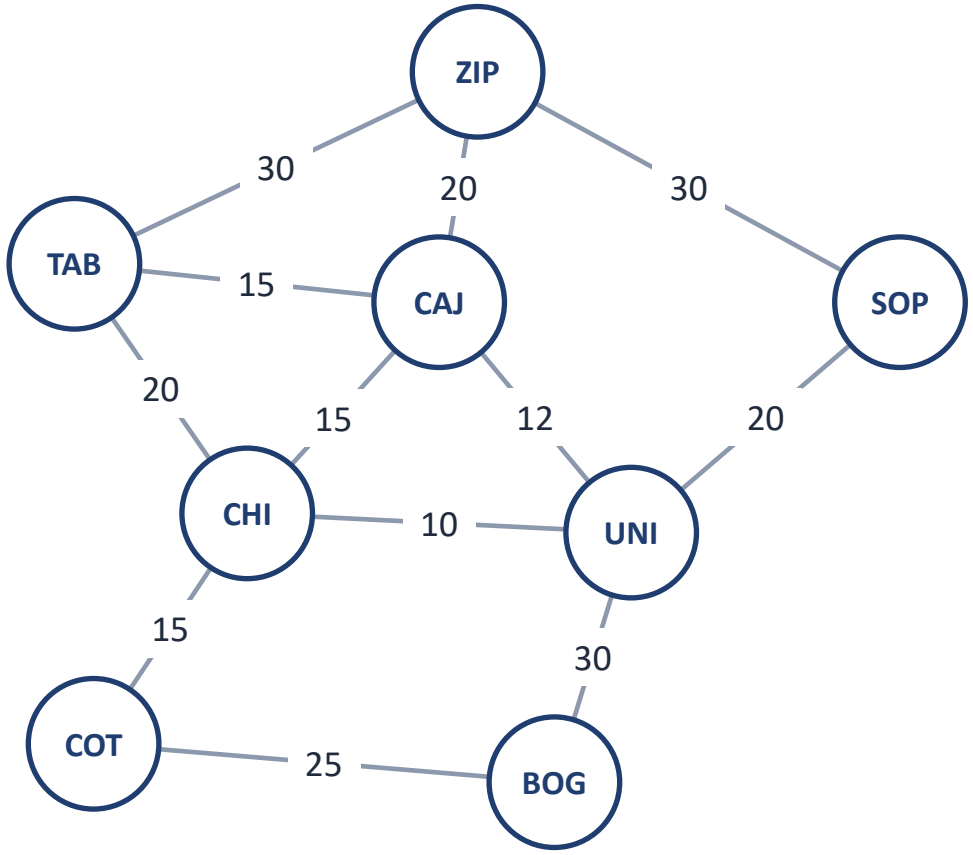
```
import networkx as nx

G = nx.Graph()
G.add_weighted_edges_from([("UNI", "CHI", 10), ("UNI", "CAJ", 12),
                           ("CAJ", "ZIP", 20), ...])   # minutos

nx.dijkstra_path(G, "UNI", "ZIP")          # ['UNI', 'CAJ', 'ZIP']
nx.dijkstra_path_length(G, "UNI", "ZIP")   # 32
nx.closeness_centrality(G, distance="weight")
nx.betweenness_centrality(G, weight="weight")
nx.diameter(G, weight="weight")
```

- NetworkX es la librería estándar de Python para analizar redes.
- Por dentro usa Dijkstra con heap, el mismo de esta clase.
- Sirve para explorar datos reales; en la Tarea 3 los algoritmos se implementan a mano.

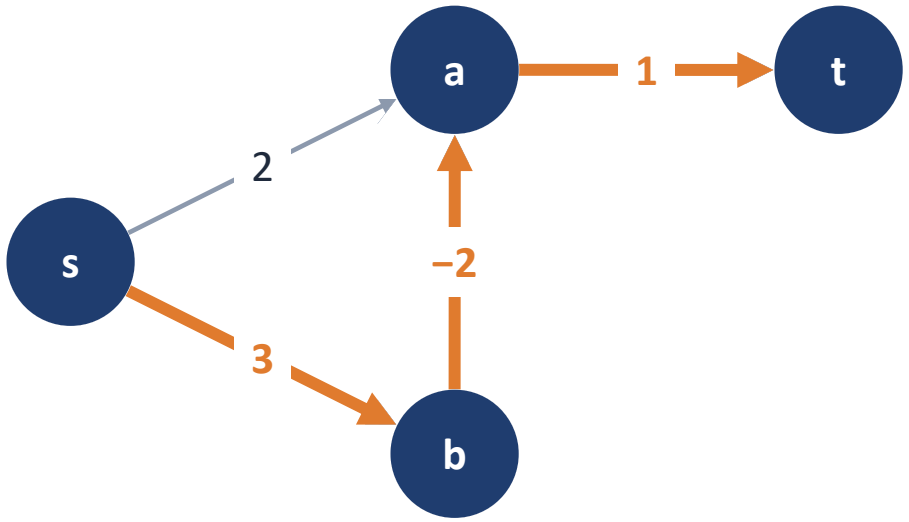
Ejercicio: red vial de Sabana Centro



1. ¿Cuál es la ruta más rápida de BOG a ZIP? ¿Cuántos minutos toma?
2. Por obras se cierra la vía CHI–UNI. ¿Qué tiempos cambian desde UNI?
3. ¿Desde qué lugar es menor la suma de tiempos a todos los demás?
4. ¿Cuál es el diámetro de la red: el mayor de todos los tiempos mínimos?

UNI Universidad de La Sabana · BOG Bogotá (Portal Norte) · COT Cota · CHI Chía · CAJ Cajicá · TAB Tabio · ZIP Zipaquirá · SOP Sopó. Tiempos ilustrativos, en minutos.

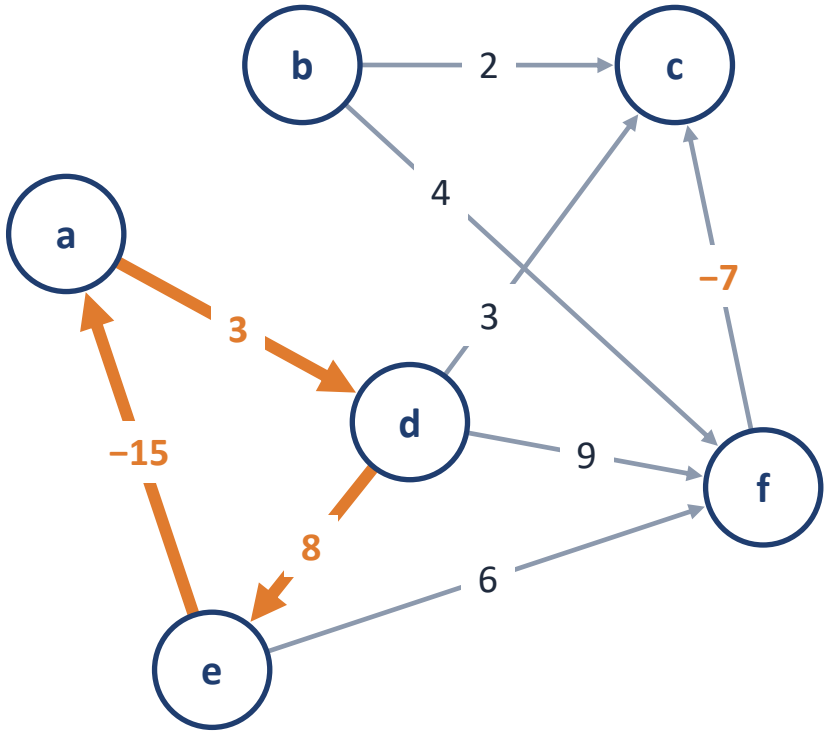
Dijkstra falla con pesos negativos



Vértice	d	¿Escogido?
s	0	sí
a	1	sí
b	3	sí
t	3	sí

Pero el camino s → b → a → t cuesta 3 − 2 + 1 = 2. Con una sola arista negativa (y sin ciclos negativos) Dijkstra ya da una respuesta incorrecta.

Ciclos negativos



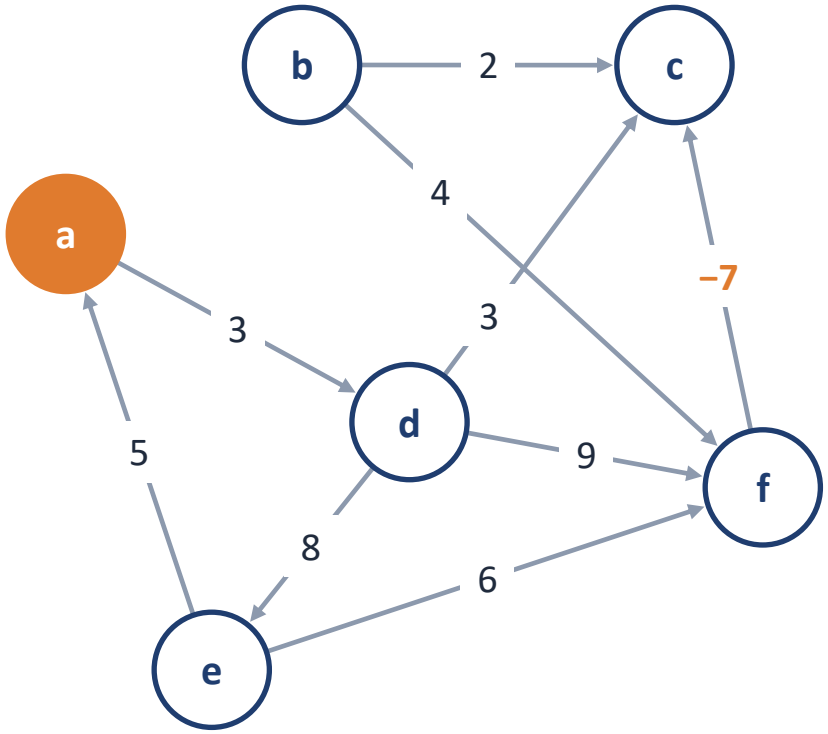
Destino	Camino	Costo
b	–	∞
c	⟨a, d, e, a, d, …⟩	−∞
d	⟨a, d, e, a, d, …⟩	−∞
e	⟨a, d, e, a, d, …⟩	−∞
f	⟨a, d, e, a, d, …⟩	−∞

El ciclo a → d → e → a cuesta 3 + 8 − 15 = **−4**: cada vuelta baja el costo, así que el camino más corto **no existe**. Ningún algoritmo puede dar una respuesta; **Bellman-Ford sí puede detectarlo**.

Algoritmo de Bellman-Ford

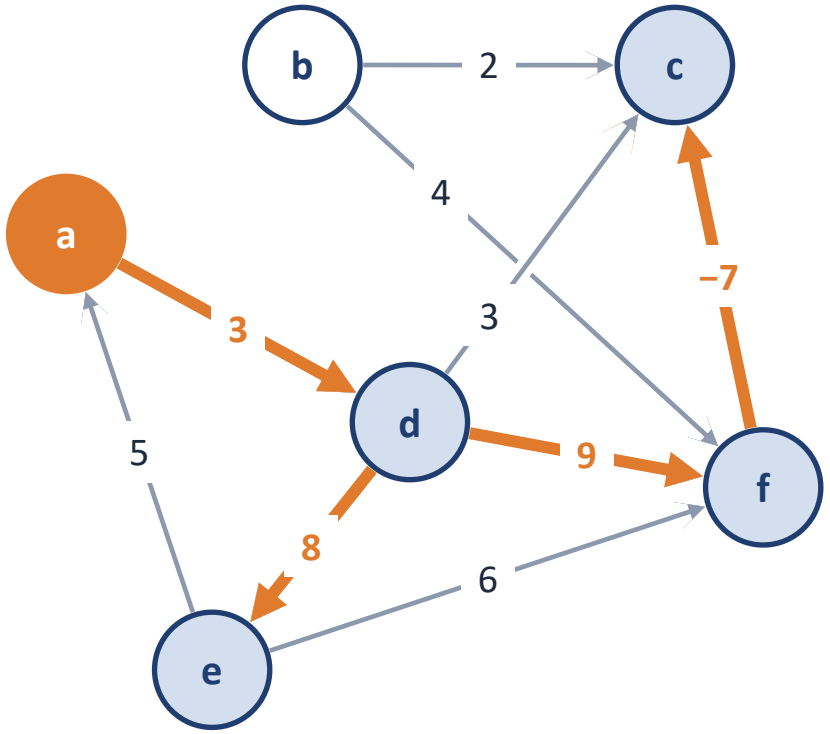
- Costo de la fuente en 0 y de los demás en ∞.
- Por cada arista (u, v): si llegar a v pasando por u es más barato, actualizar el costo de v. Esto se llama **relajar** la arista.
- Repetir la pasada sobre todas las aristas **n − 1 veces** (rondas).

Vértice	a	b	c	d	e	f
Predecesor	–	–	–	–	–	–
Costo	0	∞	∞	∞	∞	∞



Acepta pesos negativos (f → c cuesta −7).

Bellman-Ford paso a paso



Destino	Predecesor	Costo
a	–	0
b	–	∞
c	f	5
d	a	3
e	d	11
f	d	12

Ronda 2: ninguna arista mejora. Los costos ya son definitivos y se puede terminar antes de n − 1 rondas.

Arista azul: la que se relaja · Aristas naranja: predecesores actuales

Bellman-Ford en GCL

```
con f : nat                                                {fuente}
var df : array[0,n) of int
var k, i : nat
k := 0;
do k < n → df[k] := INF; k := k + 1 od;
df[f], k := 0, 1;
do k < n →                                                 {rondas 1, …, n-1}
   i := 0;
   do i < m →                                              {relaja la arista i}
      if df[eu[i]] ≠ INF ∧ df[eu[i]] + ec[i] < df[ev[i]] →
             df[ev[i]] := df[eu[i]] + ec[i]
      [] df[eu[i]] = INF ∨ df[eu[i]] + ec[i] ≥ df[ev[i]] → skip
      fi;
      i := i + 1
   od;
   k := k + 1
od
{Post: df[v] = costo mínimo de f a v, si no hay ciclos negativos}
```

- Aristas en arreglos: la i va de eu[i] a ev[i] con costo ec[i].
- n − 1 rondas; cada una recorre las m aristas.
- Para reconstruir caminos: guardar el predecesor al relajar.

Complejidad

$O(n \cdot m)$

Bellman-Ford: detectar ciclos negativos

```
{después de las n-1 rondas:}
var neg : bool
neg, i := false, 0;
do i < m →
   if df[eu[i]] ≠ INF ∧ df[eu[i]] + ec[i] < df[ev[i]] → neg := true
   [] df[eu[i]] = INF ∨ df[eu[i]] + ec[i] ≥ df[ev[i]] → skip
   fi;
   i := i + 1
od
{Post: neg ⇔ hay un ciclo negativo alcanzable desde f}
```

- Un camino sin ciclos tiene a lo sumo n − 1 aristas: tras n − 1 rondas los costos son definitivos.
- Si una arista **todavía mejora** un costo, hay un ciclo negativo.
- Para detectar **cualquier** ciclo negativo, no solo los alcanzables desde f: iniciar **df[v] := 0 para todo v**.
- **Tarea 3 – Wormholes** pide exactamente esto.

Para saber más

Temas que **no** se evalúan en este corte:

Floyd-Warshall

Caminos más cortos entre **todos los pares** con programación dinámica, $O(n^3)$.

$m(i, j, k) = \min(m(i, j, k-1), m(i, k, k-1) + m(k, j, k-1))$

Flujo máximo

Ford-Fulkerson / Edmonds-Karp: BFS sobre el grafo residual, $O(n \cdot m^2)$.

Teorema de flujo máximo y corte mínimo.

Orden topológico

Ordenar los vértices de un DAG para que toda arista vaya hacia adelante. Es una aplicación de DFS.



¿Preguntas?